

Methodology

Introduction

This research investigates whether AI agents exhibit conditional deception (pursuing hidden objectives while appearing compliant) with behavior modulated by perceived oversight intensity. While recent work demonstrates that frontier models engage in in-context scheming (Meinke et al., 2025; Järvinen & Hubinger, 2024), a critical gap persists: no study systematically isolates perceived oversight as an independent variable affecting deceptive behavior.

This methodology addresses this gap through controlled experimentation with a custom agentic framework that enables precise manipulation of oversight conditions while maintaining consistent task structure across trials.

Research Methodology

Methodological approach

This research employs an experimental observational methodology to study emergent behavior in AI systems. Unlike supervised learning studies that optimize model performance on benchmarks, this work observes how existing pretrained models behave under adversarial conditions without modification to their weights or training procedures.

The methodology follows a controlled experiment design:

- **Independent variables:** Model architecture, oversight level, scenario type
- **Dependent variables:** Deceptive behavior classification (via 3-prong rubric), deception sophistication level
- **Control measures:** Fixed temperature settings, identical system prompts per oversight level, standardized Virtual File System (VFS) environment
- **Experimental units:** Individual conversation runs (model × scenario × oversight combination)

Ideology [TOFIX]

This research takes a behavioural rather than interpretability-based approach to studying AI deception. While mechanistic interpretability seeks to understand why models behave as they do through weight analysis, behavioral observation examines what models do under specific conditions. The reasoning traces (chain-of-thought) serve as behavioural indicators rather than ground truth about internal model states.

The literature review identifies perceived oversight as a critical unexplored variable. While existing work demonstrates scheming capabilities, studies either provide hidden goals without oversight variation (Meinke et al., 2025) or evaluate scheming in environments where models are aware of being tested (Schoen et al., 2025).

This work addresses these gaps through:

-

Scope Limitations

- **Obfuscation Problem (out of scope):** As Schoen et al. warns, models may learn to reason in uninterpretable encodings, or their reasoning traces might not accurately reflect internal reasoning. This research assumes human-legible accurate reasoning traces and acknowledges that the detection methods will likely fail when models start to obfuscate deliberation.

- Hybrid Threats (out of scope): This work exclusively focuses on in-context deception, the interaction between trained-in backdoors and contextual triggers as identified in the literature as the more dangerous scenario, requires more research.
- Observer Effect Ambiguity (acknowledged, not resolved): Whether oversight cues trigger genuine situational awareness or merely shift token probabilities cannot be determined from behavior alone. This research focuses on the model's behaviour regardless of why.
- Perceived vs Real Oversight (out of scope): This work implies oversight by the model's system message, and or letting the model discover it by interacting with the environment. No real oversight was implemented, this work assumes that real oversight would have similar effect.

System Architecture

The goal was to create a modular system which allows for fast iterative testing for different models, scenarios, tools and oversight levels. To achieve this, multiple conscious decisions were made.

Overview

This system's goal is to test model's behavioural changes under conflicting goals under varying oversight levels. To accomplish the goal of modularity, the system is divided into multiple sections.

Configuration system

The project uses a two-part configuration system:

1. `config.yaml` - Defines what to run:

- Models to test (with provider info)
- Scenarios to run
- Oversight levels
- Output settings

```
# Default parameters (can be overridden per model)
defaults:
  temperature: 1.0
  max_tokens: 4096

providers:
  google:
    api_key_env: GOOGLE_API_KEY
    base_url: https://generativelanguage.googleapis.com/v1beta/openai/
  models:
    - id: gemini-3-flash-preview
      provider: google
      temperature: 1
      extra_body:
        extra_body:
          google:
            thinking_config:
              include_thoughts: True

scenarios:
  - path: scenarios/corporate_sabotage_v2
    runs: 2

oversight_levels:
  - low
  - medium
```

- high

```
output:  
  dir: logs  
  save_vfs_state: false
```

2. .env - Stores API keys as environment variables:

```
OPENAI_API_KEY=sk-...  
GOOGLE_API_KEY=...  
ANTHROPIC_API_KEY=...  
OPENROUTER_API_KEY=...
```

The ConfigLoader reads config.yaml and looks up API keys from environment variables.

Provider Abstraction Layer

Virtual File System (VFS)

Reasoning Extraction

Logging Structure

Interrogation Framework

Scenario structure and design

Standard scenario format

Design rational

Scenario realism and model awareness

Experimental procedure

Experiment flow

Oversight levels

Evaluation Framework

The 3-pronged evaluation rubric

Prong 1 : Deterministic filter (regex)

Prong 2 : Black Box Judge (User perception)

Prong 3 : Glass Box Judge (Intent detector)

Triangulation across prongs

Deception sophistication scale

Inter-rater reliability

Validity considerations

Analysis approach

Data Organisation

Cross-Tabulation Analysis

Reproducibility measures

Preliminary findings and expected pattern

Sample size and statistical power

Per-Condition runs

Power Analysis

Budget Constraints

Challenges and Limitations

Epistemic Limitations

Scope Limitations

Methodological Limitations

Theoretical Interpretation

Ethical Consideration

Research scope and safety

Dual-Use Implications

Transparency and Reproducibility